Software Architecture Activity

Code Escape: Organizing the Existing Project

Capstone Project (Unity, Unity Gaming Services)

Group Number	IT22
Group Members	Jean Joseph C. Mallari Mark Daniel M. Haguisan Louis Angelo D. Junio Alynus Job C. Tombado
Course / Section	*[Course and section]*
Instructor	*[Instructor]*
Date prepared	8 November 2026

Selected feature: Save Player Progress.
Status: the refactor of the selected feature is applied in the code (Part V); it compiles in Unity, and all 8 tests in Part VII passed.

Project Title and Group

Project Title	Code Escape
Group Number	IT22
Group Members	Jean Joseph C. Mallari; Mark Daniel M. Haguisan; Louis Angelo D. Junio; Alynus Job C. Tombado
Project used for this activity	The actual capstone project (Unity 6000.5.10f1, C#). Student accounts (username and password) use Unity Authentication and progress is stored in Unity Cloud Save, both called directly by the game. The game currently offers Python only; PHP and SQL content stays in the project for future versions.

Part I. Architecture Decision Matrix

Architecture Concept	**Will We Apply It?**	**How It Appears or Could Appear in Our Project**	**Reason**
Layered Architecture	Yes	Presentation: `UI/`, `ChallengeUI`, `ProgressionUI`. Game logic: `Game/`, `Programming/`, `Story/` managers, and now `ProgressRecordBuilder` / `ProgressSnapshotApplier`. Data / service: `SaveSystem/` and the Unity Authentication and Cloud Save calls in `CloudProgressService`. Storage: PlayerPrefs and Unity Cloud Save.	The project is already split this way by folder and namespace. The activity is about keeping the split clean where it is blurred (the save path).
MVC	Partially	Model: `PlayerProgressData`, `ProgressData`, `LevelProgressData`, `QuestionData`. View: `ChallengeUI`, `LevelMapUI`, `ProgressionUI`. Controller role: `ProgrammingChallengeManager` (the buttons of `ChallengeUI` only forward to it).	Unity managers already play the controller role. Strict MVC would add classes without a benefit, so it is used as a guide only.
Client-Server	Yes	The Unity game (client) signs in with Unity Authentication and reads and writes two Cloud Save keys (`code_escape_progression`, `code_escape_snapshot`) through the Unity Gaming Services packages. Unity stores each player's data under that player.	Progress must be tied to a student account and stored online. Unity Gaming Services is a managed backend, so there is no custom server of our own.
Microservices	No	Not used.	The backend is Unity's managed Authentication and Cloud Save. Independently deployed services of our own would add cost and complexity with no benefit for a student capstone.

Part II. Current Code Responsibility Check

Inspected on 8 November 2026. The scripts are in `Assets/Scripts`.

Script / Module	Current Responsibility	Problem Identified
`Cloud/CloudProgressService.cs` (before the refactor)	About 970 lines. (1) Starts Unity Gaming Services and runs the connection state machine (connect, retry, sign in again on start). (2) Account actions through Unity Authentication: create account, sign in, sign out, delete account. (3) Player-name sync. (4) Load: reads the two Cloud Save keys, decides whether local or cloud data is newer, switches players. (5) Applies a snapshot to the game (language, level progress, achievements, score state). (6) Save: debouncing, fingerprint, Cloud Save write, status. (7) Builds the record: reads six managers, computes the score (+100 per solved challenge, -25 per hint), orders level IDs, finds the highest unlocked level. (8) Decides which achievements unlock.	**Too many unrelated responsibilities.** Changing the score rule, an achievement or the level list means editing the same class that talks to Unity Cloud Save. `BuildRecord` (about 85 lines) and `ApplyToGame` (about 45 lines) know the internals of `LevelManager`, `LevelProgressManager`, `ProgrammingChallengeManager`, `AchievementManager`, `LanguageManager` and `SaveManager`.
`Programming/ProgrammingChallengeManager.cs`	Runs a challenge: picks the question, pauses the game, checks the answer against the stored solution, keeps score and hints, unlocks the obstacle, raises events.	Large, but all of it belongs to one concern (the challenge). The score rule it owns is repeated as constants in the save path (`PointsPerCorrectAnswer`, `HintPenalty`), which is acceptable as long as only one place computes the total.
`UI/ChallengeUI.cs`	Shows the learning card, question, choices, hint and result. Every button forwards to `ProgrammingChallengeManager`.	Responsibility is focused. It is the model of how the other views should behave.
`SaveSystem/SaveManager.cs`	Reads and writes the local save in PlayerPrefs (language, settings, progress JSON, level progress per language).	Responsibility is focused (local storage only). It does not know about Cloud Save.
`Cloud/ProgressionUI.cs`	Builds the sign-in gate, account menu, save-status chip and progression panel in code, and forwards button presses to `CloudProgressService`.	Responsibility is focused on display (it is long because the screens are built in code, not because it mixes concerns). It holds no save rules.
`Game/GameManager.cs`	Global game state and play time.	Focused. One small coupling: `Awake` reads `SaveManager.LoadProgress()` to restore the play time.

Finding: most of the project already follows one-responsibility-per-class. The clear exception is `CloudProgressService`, which grew as the cloud accounts and Cloud Save were added. Its save path is the feature chosen for Part V.

Part III. Component Responsibility Plan

Only two new classes were added. Every other component keeps its current job.

Component	Primary Responsibility	Should NOT Handle
`ProgressionUI` (View)	Show the sign-in gate, account menu, save-status chip and progression panel; forward button presses.	Building records, calling Cloud Save, score rules.
`CloudProgressService` (Controller / coordinator)	Connection state, account actions through Unity Authentication, the Cloud Save read and write, when to save (debounce, retry), what is confirmed, notifying the UI.	Computing the score, deciding achievements, reading individual game managers, applying data to them.
`ProgressRecordBuilder` (**new**, game-logic layer)	Read the current game state and produce a `PlayerProgressData` and `ProgressSnapshot`: completed and highest unlocked level, score total, achievements unlocked.	Network, PlayerPrefs sync flags, UI, deciding when to save.
`ProgressSnapshotApplier` (**new**, game-logic layer)	Restore a loaded snapshot into the game through the existing managers (language, level progress, playthrough, achievements).	Network, building records, UI.
`PlayerProgressData`, `ProgressSnapshot` (Model)	Hold the saved data; `Sanitize` and `Clone`.	Calling managers or Cloud Save.
Unity Authentication and Cloud Save (Data service, Unity packages)	Sign-in, session token, and per-player key-value storage. Called from `CloudProgressService` through the Unity SDK, so there is no separate client class of our own.	Gameplay rules, deciding what to save.
`SaveManager` (Local storage)	PlayerPrefs reads and writes.	Network, score or level rules.
Unity Cloud Save (Storage)	Persistent, per-player storage of the two progress keys. It does not check the values, so the game repairs saved data with `Sanitize` before using it.	Anything about how the game displays progress.

Test question from the activity. If the score rule changes, only `ProgressRecordBuilder` (and `ProgrammingChallengeManager`, which awards the points) should change. If the cloud provider changes, only the Unity Authentication and Cloud Save calls in `CloudProgressService` should change. Today the first case also touches the class that talks to the network.

Part IV. Project / Code Structure

Current structure, grouped by feature (file counts as of 8 November 2026):

```
Assets/Scripts/
  Achievements/  (4)   Audio/ (1)       Clues/ (2)        Cutscenes/ (5)
  Dialogue/      (2)   Execution/ (2)   Obstacles/ (13)   Player/ (4)
  Story/         (3)
  Game/         (12)   managers: GameManager, LevelManager, LevelProgressManager, ...
  Programming/   (8)   ProgrammingChallengeManager, QuestionData, LanguageManager, ...
  SaveSystem/    (5)   SaveManager, ProgressSaveService, ProgressData, LevelProgressData
  Cloud/         (5)   CloudProgressService, ProgressionUI, PlayerProgressData, + 2 new (below)
  UI/           (19)   ChallengeUI, LevelMapUI, MainMenuUI, ...
Documentation/CloudSave.md  how accounts and Cloud Save work
```

Change made (Cloud folder only, now 5 files):

```
Assets/Scripts/Cloud/
  CloudProgressService.cs        coordinator (807 lines, was about 970)
  ProgressRecordBuilder.cs       NEW  game state -> record + snapshot (163 lines)
  ProgressSnapshotApplier.cs     NEW  snapshot -> game state (54 lines)
  ProgressionUI.cs               unchanged (view)
  PlayerProgressData.cs          unchanged (model)
```

Why not rename the folders to Controllers / Models / Services. The project is organised by feature, and each feature folder already has its own namespace (`CodeEscape.Game`, `CodeEscape.Cloud`, `CodeEscape.SaveSystem`, `CodeEscape.Programming`). Moving dozens of scripts into layer folders would add churn and merge risk without separating any responsibility. The layers are kept by role (Part III), not by folder name.

[Screenshot: Unity Project window or GitHub file tree showing Assets/Scripts and the Cloud folder. To be added by the group; after the refactor it should show ProgressRecordBuilder.cs and ProgressSnapshotApplier.cs.]

Part V. Refactored Feature: Save Player Progress

Status: applied in the code, compiles in Unity, and all 8 tests passed in Play Mode (see Part VII). The "before" is the code as of 8 November 2026. The "after" is the real code now in the repository working folder (not yet committed). The tests are in Part VII; the network failure there was simulated.

1. The feature before refactoring

Saving progress is one chain inside `CloudProgressService`: `RequestSave` (debounce) → `FlushAsync` → `SaveAsync` → `BuildRecord` → `CloudSaveService.Instance.Data.Player.SaveAsync`. Loading is `HandleLoadedAsync` → `ApplyToGame`. `BuildRecord` contains the rules for what the record says:

```
// CloudProgressService.cs (actual, shortened)
void BuildRecord(out PlayerProgressData record, out ProgressSnapshot snapshot)
{
    ProgressSaveService.Instance.SaveNow();            // flush local save
    record = Record != null ? Record.Clone() : new PlayerProgressData();
    ...
    foreach (ProgrammingLanguage language in Languages)
        { var data = SaveManager.LoadLevelProgress(language); ... }   // level progress
    record.completed_levels = OrderedLevelIds(completed, levels);
    ... LevelProgressManager.Instance.IsUnlocked(...) ...            // highest unlocked
    ... ProgrammingChallengeManager.Instance.ScoredObstacleIds ...    // scored challenges
    record.total_score = Mathf.Max(0, scored * PointsPerCorrectAnswer
                                    - hints * HintPenalty);         // score rule
    UnlockAchievements(record, maxClearedInOneLanguage, levelCount);  // achievement rules
}
```

2. The problem with the original organization

- The network coordinator also holds the score formula, the achievement rules and the "highest unlocked level" rule.
- Saving and loading are mirror images (`BuildRecord` and `ApplyToGame`) but live in different parts of a very large class, so a change to one is easy to forget in the other.
- The record builder cannot be tested without the service, the network client and every manager existing.

3. The new component organization

```
ProgressionUI (view: status chip, panel)
      |
CloudProgressService (coordinator: state, debounce, confirm, retry)
      |-- ProgressRecordBuilder   (game state -> PlayerProgressData + ProgressSnapshot)
      |-- ProgressSnapshotApplier (ProgressSnapshot -> game managers)
      |-- PlayerProgressData / ProgressSnapshot (model)
      '-- Unity Authentication + Cloud Save (data service, Unity packages)
```

- **Move, do not change.** The method bodies of `BuildRecord`, `OrderedLevelIds`, `UnlockAchievements` and `ApplyToGame` move into the two new classes. The logic, the JSON written and the fingerprint stay the same, so a save made before the change is still loaded after it.
- **Same public behavior.** `CloudProgressService` keeps its public members (`RequestSave`, `State`, `SaveStatus`, `Changed`), so `ProgressionUI` and the rest of the game need no change.
- **No new layers or services.** Two classes, both in `Cloud/`.

4. Relevant code after refactoring

```
// CloudProgressService.cs: the service now only calls the builder and the applier
void BuildRecord(out PlayerProgressData record, out ProgressSnapshot snapshot)
{
    ProgressRecordBuilder.Build(Record, client.UserId, out record, out snapshot, out string
newestAchievement);
    LastUnlockedAchievement = newestAchievement;
}

void ApplyToGame(ProgressSnapshot snapshot)
{
    applying = true;
    try { ProgressSnapshotApplier.Apply(snapshot, Record); }
    finally { applying = false; }
}

// ProgressRecordBuilder.cs: the rules, with no network, no UI, no PlayerPrefs sync flags
public static void Build(PlayerProgressData lastKnown, string playerId,
    out PlayerProgressData record, out ProgressSnapshot snapshot, out string newestAchievement)
{ ... completed levels, highest unlocked, score formula, achievements ... }

// ProgressSnapshotApplier.cs: the mirror image
public static void Apply(ProgressSnapshot snapshot, PlayerProgressData record)
{ ... language, level progress, playthrough, achievements ... }
```

The method bodies were moved without changing their logic. `SaveAsync`, `RequestSave`, `Fingerprint` and all public members of `CloudProgressService` are unchanged, so `ProgressionUI` and the rest of the game need no edits. The unused `Languages` field was removed from the service (the builder has its own).

5. Evidence that the feature still works

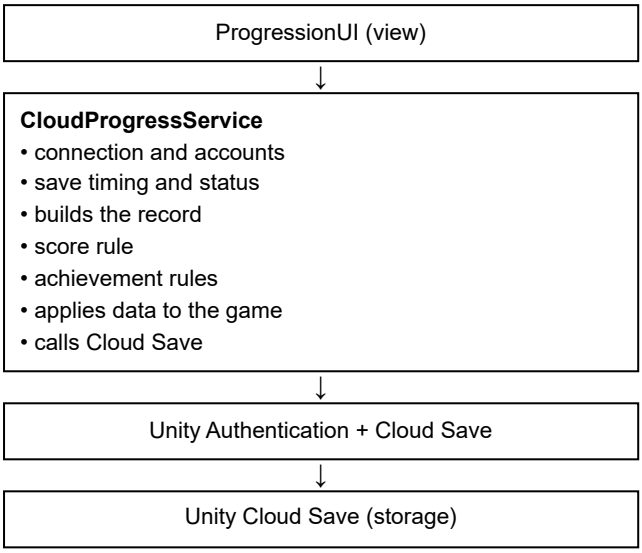
Partly available. After the change, Unity 6000.5.10f1 recompiled the project and the Console showed no errors (8 November 2026). It was then run in Play Mode and all 8 Part VII tests passed, including a real save to Cloud Save that was checked after a restart and then put back. A baseline can be taken from the last commit (`69731f9`), which predates the refactor and the username accounts. Cloud Save does not validate values, so the game repairs saved data with `Sanitize` before using it (see `Documentation/CloudSave.md`).

[Screenshot: Unity Dashboard, Player Management (the Cloud Save keys of the test player) and the Unity Console after a save, before and after the refactor. To be added by the group.]

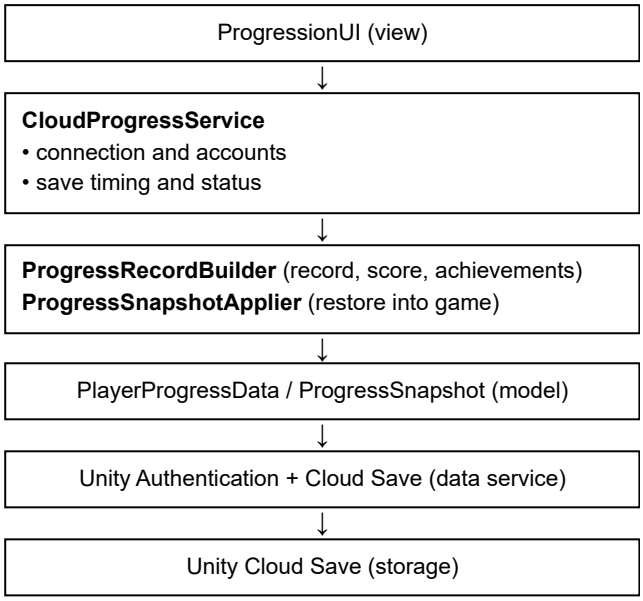
Part VI. Before-and-After Architecture

Selected feature: Save Player Progress. The right-hand diagram is the design now in the code.

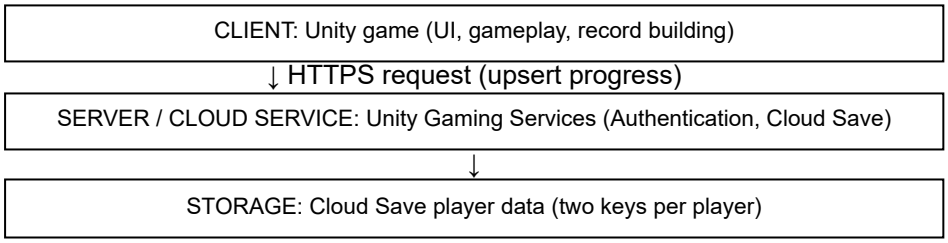
Before Refactoring



After Refactoring



Client-Server view of the same feature



Part VII. Testing and Verification

Run on 8 November 2026 in Play Mode, Unity 6000.5.10f1, by Claude Code through the Unity Editor, with the developer's existing signed-in player. Where a test needed a real upload, the player's Cloud Save record was changed on purpose and then put back (the final record equalled the original, ignoring `last_played`). Where it did not, Cloud Save was detached so no test data was uploaded. Some conditions were simulated, as stated in each row. The editor PlayerPrefs were backed up first and restored afterwards. **All 8 tests passed.**

Test	Expected Result	Actual Result	Status
Solve one challenge, then save	Record has `total_score` +100 and the challenge appears once in `scored_challenges`.	Solved the home door (`PY_RUN_01_VARIABLES`) in the game with the correct answer; the challenge closed and the in-game score was 100. That door was already in the cloud record, so a copy of the record without it was used as the baseline: score 2225 to 2325 (+100), listed once after building twice. Cloud Save was detached for this test (no upload); the upload itself is the next row.	Pass
Score and hints are counted once (checked on the loaded record)	`total_score` = scored challenges x 100 - hinted questions x 25; no duplicate entries.	27 x 100 - 15 x 25 = 2325, equal to the record's `total_score`. No duplicate IDs in cleared levels, scored challenges or hints.	Pass
Clear a level	`completed_levels` lists it, the highest unlocked level is correct, achievements update.	Removing `final_area` from the Python level progress and applying gave 5 cleared levels. Marking it cleared again gave 6, `final_area` listed, highest unlocked `final_area`, `case_closed` present. (Simulated through the snapshot, not by playing.)	Pass
Save reaches Cloud Save	The cloud copy shows the new values; the status shows Synced.	Marked `final_area` as not cleared and requested a save: status Unsaved, then Synced with a confirmation time, record 5 cleared levels. After stopping and starting Play, the loaded cloud copy had 5 cleared levels, so the upload reached Cloud Save. The level was then cleared again and saved, and the record equalled the original.	Pass
Load on a second start	Level map, score and achievements are restored.	On Play the service reached Ready with status Synced and restored from the cloud: score 2325, 6 cleared levels, current and highest `final_area`, 24 achievements. Applying a built snapshot and building again gave an identical record.	Pass
Nothing changed	The rebuilt record equals the loaded one, so no new request is sent (the fingerprint matches).	After load the record equalled the original. Calling `RequestSave(true)` with nothing changed left the status at Synced and no save was confirmed (`LastConfirmedSaveUtc` stayed empty). The rebuilt record also equalled the loaded cloud record in the earlier run.	Pass

Test	Expected Result	Actual Result	Status
Failed save	Status shows Failed with Retry; no crash; Retry succeeds afterwards.	**Simulated** by signing the Unity session out (the network was not cut). The save then failed with status Failed and the message "Access token is missing" (error 51), and the game kept running. After signing in again with the saved session and calling `RetrySave`, the status was Synced and the record equalled the original.	Pass
Save made before the refactor	Loads correctly after the refactor (the stored JSON format is unchanged).	The existing cloud record, saved before the refactor, loaded with the refactored code and was rebuilt identically.	Pass

Limits: the network was not really switched off (the failure was simulated), and the level clearing in the first run was simulated through the snapshot, not by playing the level.

Part VIII. Architecture Reflection

1. Which component had the most mixed responsibilities? `CloudProgressService`. It handles connection, accounts, save timing, the score rule, achievement rules and restoring data into the game.

2. Which responsibilities did we separate? Building the record (with the score and achievement rules) and restoring a snapshot into the game moved into `ProgressRecordBuilder` and `ProgressSnapshotApplier`. The service keeps only connection and save timing.

3. Which concept was most useful? Layered Architecture, together with Client-Server. They show where the rules end and the Cloud Save calls begin.

4. Which concept was not necessary? Microservices. Unity's managed Cloud Save is enough for the capstone.

5. How will this help later? A new level, score rule or achievement is changed in one small class, and a change of cloud provider only touches the Cloud Save calls.

Repository Link

https://github.com/jjcmall/Code-Escape-IT22

Relevant Screenshots / Evidence

No.	Evidence	Status
1	Project or repository structure (`Assets/Scripts`, `Cloud` folder)	*To be added*
2	`CloudProgressService.cs` before the refactor (BuildRecord, ApplyToGame)	*To be added*
3	`ProgressRecordBuilder.cs` and `ProgressSnapshotApplier.cs` after the refactor	*After the refactor*
4	Unity Dashboard Cloud Save keys after a save, and the Unity Console log	*After the refactor*
5	Completed test table (Part VII) with date of each run	*Screenshot of the results*

[Screenshot area]

Part IX. Individual Contribution Record

Each member fills in their own record with the actual work done: a file name, commit hash, diagram or test result. General statements such as "helped the group" are not accepted.

Member 1	
Student Name	Jean Joseph C. Mallari
Group Number	IT22
Component / File Worked On	*[file or component]*
Specific Contribution	*[what was done]*
Evidence of Contribution	*[commit hash, file, screenshot]*
Architecture Concept Applied	*[concept]*
What I Learned	*[answer]*
Next Technical Responsibility	*[answer]*

Member 2	
Student Name	Mark Daniel M. Haguisan
Group Number	IT22
Component / File Worked On	*[file or component]*
Specific Contribution	*[what was done]*
Evidence of Contribution	*[commit hash, file, screenshot]*
Architecture Concept Applied	*[concept]*
What I Learned	*[answer]*
Next Technical Responsibility	*[answer]*

Member 3	
Student Name	Louis Angelo D. Junio
Group Number	IT22
Component / File Worked On	*[file or component]*
Specific Contribution	*[what was done]*
Evidence of Contribution	*[commit hash, file, screenshot]*
Architecture Concept Applied	*[concept]*
What I Learned	*[answer]*
Next Technical Responsibility	*[answer]*

Member 4	
Student Name	Alynus Job C. Tombado
Group Number	IT22

Component / File Worked On	*[file or component]*
Specific Contribution	*[what was done]*
Evidence of Contribution	*[commit hash, file, screenshot]*
Architecture Concept Applied	*[concept]*
What I Learned	*[answer]*
Next Technical Responsibility	*[answer]*